| S.No: 1 | Exp. Name: ***Program to implement AVL tree and its operations*** | **Date: 2026-08-20** |
|---|---|---|

**Aim:**

Write a program to create an AVL tree using a provided set of elements stored in a file. Implement operations for inserting and deleting elements from the AVL tree. After constructing the tree, output the contents of the tree in in-order traversal to a new file.

**Note:** Refer to visible test cases for better understanding.

**Source Code:**

avl_tree.c

```
// Write your code here..
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
typedef struct Node {
    int key;
    struct Node* left;
    struct Node* right;
    int height;
} Node;
// Utility functions
int height(Node* N) {
    if (N == NULL) return 0;
    return N->height;
}
/*The max() function helps us find which subtree (left or right) is
taller. Since the height of a node depends on the longest path from
that node to a leaf, we take the maximum of the left and right subtree
heights*/
int max(int a, int b) {
    return (a > b) ? a : b;
}

Node* newNode(int key) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->key = key;
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}
//example insert 50(50 is key ,its height is initialized with 1)
// Rotations
//LL imbalance
Node* rightRotate(Node* y) {
    Node* x = y->left;
    Node* T2 = x->right;
    x->right = y;
    y->left = T2;
    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;
    return x;
}
Node* leftRotate(Node* x) {
```

```
    Node* y = x->right;
    Node* T2 = y->left;
    y->left = x;
    x->right = T2;
    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;
    return y;
}
int getBalance(Node* N) {
    if (N == NULL) return 0;
    return height(N->left) - height(N->right);
}
// Insert
Node* insert(Node* node, int key) {
    if (node == NULL)
        return newNode(key);
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else
        return node; // No duplicates
    node->height = 1 + max(height(node->left), height(node->right));
    int balance = getBalance(node);
    // Balancing
//LL IMBALANCE
    if (balance > 1 && key < node->left->key) return rightRotate(node);
//RR imbalance
    if (balance < -1 && key > node->right->key) return
leftRotate(node);
//LR IMBALANCE
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
//RL IMBALANCE
    if (balance < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}
// Minimum value node
Node* minValueNode(Node* node) {
    Node* current = node;
    while (current->left)
```

```
        current = current->left;
    return current;
}
// Delete
Node* deleteNode(Node* root, int key, int* found) {
    if (!root) return root;
    if (key < root->key)
        root->left = deleteNode(root->left, key, found);
    else if (key > root->key)
        root->right = deleteNode(root->right, key, found);
    else {
        *found = 1;
        if (!root->left || !root->right) {
            Node* temp = root->left ? root->left : root->right;
            if (!temp) {
                temp = root;
                root = NULL;
            } else
                *root = *temp;
            free(temp);
        } else {
            Node* temp = minValueNode(root->right);
            root->key = temp->key;
            root->right = deleteNode(root->right, temp->key, found);
        }
    }
    if (!root) return root;
    root->height = 1 + max(height(root->left), height(root->right));
    int balance = getBalance(root);
    if (balance > 1 && getBalance(root->left) >= 0) return
rightRotate(root);
    if (balance > 1 && getBalance(root->left) < 0) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }
    if (balance < -1 && getBalance(root->right) <= 0) return
leftRotate(root);
    if (balance < -1 && getBalance(root->right) > 0) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }
    return root;
}
// In-order traversal
void inorder(Node* root, FILE* out) {
    if (root) {
```

```
        inorder(root->left, out);
        fprintf(out, "%d ", root->key);
        inorder(root->right, out);
    }
}
void printInorder(Node* root) {
    FILE* temp = tmpfile();
    inorder(root, temp);
    rewind(temp);
    int x;
    while (fscanf(temp, "%d", &x) == 1) {
        printf("%d ", x);
    }
    printf("\n");
    fclose(temp);
}
// Main Program
int main() {
    Node* root = NULL;
    char inputFile[20];
    printf("Input file name: ");
    scanf("%s", inputFile);
//lets say input file contains 10 20 30 40 50
    FILE* in = fopen(inputFile, "r");
    if (!in) {
        printf("Error opening input file\n");
        return 1;
    }
    int val;
        char line[256];
        fgets(line, sizeof(line), in);
        while (fscanf(in, "%d", &val) != EOF)
        root = insert(root, val);
    fclose(in);
    printf("In-order traversal:\n");
    printInorder(root);
    int choice, key;
    while (1) {
        printf("1. Insert\n2. Delete\n3. Write to file and
exit\nChoice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Element: ");
                scanf("%d", &key);
                root = insert(root, key);
```

```
                printf("In-order traversal:\n");
                printInorder(root);
                break;
            case 2:
                printf("Element: ");
                scanf("%d", &key);
                    int found = 0;
                    root = deleteNode(root, key, &found);
                    if (found)
                        printf("Deleted %d\n", key);
                    else
                        printf("%d not found\n", key);
                    printf("In-order traversal:\n");
                    printInorder(root);
                    break;
            case 3:{
                    char outputFile[100];
                    printf("Output file name: ");
                    scanf("%s", outputFile);
                    FILE* out = fopen(outputFile, "w");
                    if (!out) {
                        printf("Error opening output file\n");
                        return 1;
                    }
                    inorder(root, out);
                    fprintf(out, "\n");
                    fclose(out);
                    printf("AVL Tree written to file\n");
                    return 0;}
                default:
                printf("Invalid choice\n");
        }
    }
}
```

input1.txt

```
5
1 2 3 4 5
```

input2.txt

```
6
12 24 36 48 60 72
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
|---|
| **User Output** |
| Input file name: |
| inputfile1.txt |
| Error opening input file |

| Test Case - 2 |
|---|
| **User Output** |
| Input file name: |
| input1.txt |
| In-order traversal: |
| 1 2 3 4 5 |
| 1. Insert |
| 2. Delete |
| 3. Write to file and exit |
| Choice: |
| 1 |
| Element: |
| 10 |
| In-order traversal: |
| 1 2 3 4 5 10 |
| 1. Insert |
| 2. Delete |
| 3. Write to file and exit |
| Choice: |
| 2 |
| Element: |
| 100 |
| 100 not found |
| In-order traversal: |

| |
|---|
| 1 2 3 4 5 10 |
| 1. Insert |
| 2. Delete |
| 3. Write to file and exit |
| Choice: |
| 2 |
| Element: |
| 3 |
| Deleted 3 |
| In-order traversal: |
| 1 2 4 5 10 |
| 1. Insert |
| 2. Delete |
| 3. Write to file and exit |
| Choice: |
| 3 |
| Output file name: |
| myoutput.txt |
| AVL Tree written to file |

| **Test Case - 3** |
|---|
| **User Output** |
| Input file name: |
| input2.txt |
| In-order traversal: |
| 12 24 36 48 60 72 |
| 1. Insert |
| 2. Delete |
| 3. Write to file and exit |
| Choice: |
| 1 |
| Element: |
| 108 |
| In-order traversal: |
| 12 24 36 48 60 72 108 |
| 1. Insert |
| 2. Delete |
| 3. Write to file and exit |
| Choice: |

```
Element:
120
In-order traversal:
12 24 36 48 60 72 108 120
1. Insert
2. Delete
3. Write to file and exit
Choice:
1
Element:
144
In-order traversal:
12 24 36 48 60 72 108 120 144
1. Insert
2. Delete
3. Write to file and exit
Choice:
2
Element:
36
Deleted 36
In-order traversal:
12 24 48 60 72 108 120 144
1. Insert
2. Delete
3. Write to file and exit
Choice:
2
Element:
48
Deleted 48
In-order traversal:
12 24 60 72 108 120 144
1. Insert
2. Delete
3. Write to file and exit
Choice:
10
Invalid choice
1. Insert
2. Delete
```

| 3. Write to file and exit |
| --- |
| Choice: |
| 2 |
| Element: |
| 1440 |
| 1440 not found |
| In-order traversal: |
| 12 24 60 72 108 120 144 |
| 1. Insert |
| 2. Delete |
| 3. Write to file and exit |
| Choice: |
| 15 |
| Invalid choice |
| 1. Insert |
| 2. Delete |
| 3. Write to file and exit |
| Choice: |
| 3 |
| Output file name: |
| outputfile.txt |
| AVL Tree written to file |

| S.No: 2 | Exp. Name: ***Operation of B-Tree - Insertion, Deletion and Search*** | **Date: 2026-09-24** |
|---|---|---|

**Aim:**

Write a C program to construct a B-Tree of a given minimum degree. First, insert a list of keys into the B-Tree and print the inorder traversal after all insertions. Then, delete a given set of keys from the B-Tree and again print the inorder traversal. Finally, search for a specific key in the current B-Tree and print whether it is found or not.

**Input Format:**

- The first line contains a single integer $t$, the minimum degree of the B-Tree.
- The second line contains an integer $n$, representing the number of keys to be inserted.
- The third line contains $n$ space-separated integers — the keys to be inserted into the B-Tree.
- The fourth line contains an integer $d$, the number of keys to delete.
- The fifth line contains $d$ space-separated integers — the keys to be deleted from the B-Tree.
- The sixth line contains a single integer $k$ — the key to be searched in the B-Tree.

**Output Format:**

- The first line of output should display the inorder traversal of the B-Tree after all insertions, with each key separated by a space.
- The second line should display the inorder traversal of the B-Tree after all deletions, with each key separated by a space.
- The third line should print **Found** if the search key exists in the B-Tree. If the key is not present, print **Not Found** instead.

**Note:** Refer to the test cases for better understanding.

**Source Code:**

Btreemain.c

```
// write your code here..
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct BTreeNode
{
int *keys;
int t;
struct BTreeNode **C;
int n;
bool leaf;
}
BTreeNode;
BTreeNode* createNode(int t, bool leaf)
{
        BTreeNode* node = (BTreeNode*)malloc(sizeof(BTreeNode));
        node->t = t;
        node->leaf = leaf;
        node->keys = (int*)malloc((2 * t - 1) * sizeof(int));
        node->C = (BTreeNode**)malloc((2 * t) * sizeof(BTreeNode*));
        node->n = 0;
        return node;
}
void inorder(BTreeNode* root)
{
        if (!root) return;
        for (int i = 0; i < root->n; i++)
        {
                if (!root->leaf)
                        inorder(root->C[i]);
                printf("%d ", root->keys[i]);
        }
        if (!root->leaf)
                inorder(root->C[root->n]);
}
bool search(BTreeNode* root, int k)
{
        int i = 0;
        while (i < root->n && k > root->keys[i])
                i++;
        if (i < root->n && root->keys[i] == k)
                return true;
        if (root->leaf)
```

```
                return false;
        return search(root->C[i], k);
}
void splitChild(BTreeNode* x, int i)
{
        int t = x->t;
        BTreeNode* y = x->C[i];
        BTreeNode* z = createNode(t, y->leaf);
        z->n = t - 1;
        for (int j = 0; j < t - 1; j++)
                z->keys[j] = y->keys[j + t];
        if (!y->leaf)
        {
                for (int j = 0; j < t; j++)
                        z->C[j] = y->C[j + t];
        }
        y->n = t - 1;
        for (int j = x->n; j >= i + 1; j--)
                x->C[j + 1] = x->C[j];
        x->C[i + 1] = z;
        for (int j = x->n - 1; j >= i; j--)
                x->keys[j + 1] = x->keys[j];
        x->keys[i] = y->keys[t - 1];
        x->n++;
}
void insertNonFull(BTreeNode* x, int k)
{
        int i = x->n - 1;
        if (x->leaf)
        {
                while (i >= 0 && k < x->keys[i])
                {
                        x->keys[i + 1] = x->keys[i];
                        i--;
                }
                x->keys[i + 1] = k;
                x->n++;
        }
        else {
                while (i >= 0 && k < x->keys[i])
                        i--;
                i++;
                if (x->C[i]->n == 2 * x->t - 1)
                {
                        splitChild(x, i);
                        if (k > x->keys[i])
```

```
                                i++;
                }
                insertNonFull(x->C[i], k);
        }
}
void insert(BTreeNode** rootRef, int k)
{
        BTreeNode* r = *rootRef;
        if (r->n == 2 * r->t - 1)
        {
                BTreeNode* s = createNode(r->t, false);
                s->C[0] = r;
                splitChild(s, 0);
                int i = (k > s->keys[0]) ? 1 : 0;
                insertNonFull(s->C[i], k);
                *rootRef = s;
        }
        else {
                insertNonFull(r, k);
        }
}
int getPred(BTreeNode* x)
{
        while (!x->leaf)
                x = x->C[x->n];
        return x->keys[x->n - 1];
}
int getSucc(BTreeNode* x)
{
        while (!x->leaf)
                x = x->C[0];
        return x->keys[0];
}
void merge(BTreeNode* x, int idx)
{
        BTreeNode* c1 = x->C[idx];
        BTreeNode* c2 = x->C[idx + 1];
        int t = x->t;
        c1->keys[t - 1] = x->keys[idx];
        for (int i = 0; i < c2->n; ++i)
                c1->keys[i + t] = c2->keys[i];
        if (!c1->leaf)
        {
                for (int i = 0; i <= c2->n; ++i)
                        c1->C[i + t] = c2->C[i];
        }
```

```
        for (int i = idx + 1; i < x->n; ++i)
                x->keys[i - 1] = x->keys[i];
        for (int i = idx + 2; i <= x->n; ++i)
                x->C[i - 1] = x->C[i];
        c1->n += c2->n + 1;
        x->n--;
        free(c2);
}
void fill(BTreeNode* x, int idx)
{
        int t = x->t;
        if (idx > 0 && x->C[idx - 1]->n >= t)
        {
                BTreeNode* child = x->C[idx];
                BTreeNode* sibling = x->C[idx - 1];
                for (int i = child->n - 1; i >= 0; --i)
                        child->keys[i + 1] = child->keys[i];
                if (!child->leaf)
                {
                        for (int i = child->n; i >= 0; --i)
                                child->C[i + 1] = child->C[i];
                }
                child->keys[0] = x->keys[idx - 1];
                if (!child->leaf)
                        child->C[0] = sibling->C[sibling->n];
                x->keys[idx - 1] = sibling->keys[sibling->n - 1];
                child->n += 1;
                sibling->n -= 1;
        }
        else if (idx < x->n && x->C[idx + 1]->n >= t)
        {
                BTreeNode* child = x->C[idx];
                BTreeNode* sibling = x->C[idx + 1];
                child->keys[child->n] = x->keys[idx];
                if (!child->leaf)
                        child->C[child->n + 1] = sibling->C[0];
                x->keys[idx] = sibling->keys[0];
                for (int i = 1; i < sibling->n; ++i)
                        sibling->keys[i - 1] = sibling->keys[i];
                if (!sibling->leaf)
                {
                        for (int i = 1; i <= sibling->n; ++i)
                                sibling->C[i - 1] = sibling->C[i];
                }
                child->n += 1;
                sibling->n -= 1;
```

```
        }
        else {
                if (idx != x->n)
                        merge(x, idx);
                else
                        merge(x, idx - 1);
        }
}
void removeFromNode(BTreeNode* x, int k)
{
        int idx = 0;
        while (idx < x->n && x->keys[idx] < k)
                ++idx;
        if (idx < x->n && x->keys[idx] == k)
        {
                if (x->leaf)
                {
                        for (int i = idx + 1; i < x->n; ++i)
                                x->keys[i - 1] = x->keys[i];
                        x->n--;
                }
                else {
                        if (x->C[idx]->n >= x->t)
                        {
                                int pred = getPred(x->C[idx]);
                                x->keys[idx] = pred;
                                removeFromNode(x->C[idx], pred);
                        }
                        else if (x->C[idx + 1]->n >= x->t)
                        {
                                int succ = getSucc(x->C[idx + 1]);
                                x->keys[idx] = succ;
                                removeFromNode(x->C[idx + 1], succ);
                        }
                        else {
                                merge(x, idx);
                                removeFromNode(x->C[idx], k);
                        }
                }
        }
        else {
                if (x->leaf)
                        return;
                bool flag = (idx == x->n);
                if (x->C[idx]->n < x->t)
                        fill(x, idx);
```

```
                if (flag && idx > x->n)
                        removeFromNode(x->C[idx - 1], k);
                else
                        removeFromNode(x->C[idx], k);
        }
}
void deleteKey(BTreeNode** rootRef, int k)
{
        BTreeNode* r = *rootRef;
        removeFromNode(r, k);
        if (r->n == 0)
        {
                BTreeNode* tmp = r;
                if (r->leaf)
                        *rootRef = NULL;
                else
                        *rootRef = r->C[0];
                free(tmp);
        }
}
int main()
{
        int t;
        scanf("%d", &t);
        BTreeNode* root = createNode(t, true);
        int n, val;
        scanf("%d", &n);
        for (int i = 0; i < n; i++)
        {
                scanf("%d", &val);
                insert(&root, val);
        }
        inorder(root);
        int d;
        scanf("%d", &d);
        for (int i = 0; i < d; i++)
        {
                scanf("%d", &val);
                deleteKey(&root, val);
        }
        inorder(root);
        int target;
        scanf("%d\n", &target);
        if (search(root, target))
                printf("Found\n");
        else
```

```
                printf("Not Found\n");
        return 0;
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
|---|
| **User Output** |
| 2 |
| 7 |
| 10 20 5 6 12 30 7 |
| 3 |
| 6 30 100 |
| 12 |
| 5 6 7 10 12 20 30 |
| 5 7 10 12 20 |
| Found |

| Test Case - 2 |
|---|
| **User Output** |
| 2 |
| 6 |
| 8 3 10 1 6 14 |
| 2 |
| 1 3 |
| 2 |
| 1 3 6 8 10 14 |
| 6 8 10 14 |
| Not Found |


ID: 25B01A1228

Shri Vishnu Engineering College for Women (Autonomous)

| S.No: 3 | Exp. Name: ***Min Heap Operations*** | **Date: 2026-08-20** |
|---|---|---|



**Aim:**

Write a C program to implement a Min Heap of integers. The program should first read a list of integers and insert them into the heap while maintaining the min-heap property. It should then read the number of deletions to perform, delete the root (minimum element) that many times, and display the heap both before and after the deletions.

**Input Format:**

- The first line contains an integer $n$, representing the number of elements to insert into the heap.
- The second line contains $n$ space-separated integers — the values to insert into the heap.
- The third line contains an integer $k$, representing the number of times the root (minimum element) should be deleted from the heap.

**Output Format:**

- The first line displays the space-separated heap contents after all insertions.
- The second line displays the space-separated heap contents after performing $k$ deletions of the root element.

**Note:** Refer to the test cases for better understanding.

**Source Code:**

minheap.c

```
// write your code here...
#include<stdio.h>
#define MAX_SIZE 100
int heap[MAX_SIZE];
int size = 0;
void swap(int* a,int* b)
{
        int temp = *a;
        *a = *b;
        *b = temp;
}
void heapifyUP(int index)
{
        while(index>0)
                {
                        int parent = (index - 1)/2;
                        if(heap[index]<heap[parent])
                        {
                                swap(&heap[index],&heap[parent]);
                                index = parent;
                        }
                        else{
                                break;
                        }
                }
}
void insert(int value)
{
        heap[size] = value;
        heapifyUP(size);
        size++;
}
void heapifyDown(int index)
{
        while(1)
                {
                        int left = 2* index + 1;
                        int right = 2* index + 2;
                        int smallest = index;
                        if(left < size && heap[left]<heap[smallest])
                                smallest = left;
                        if(right < size && heap[right]<heap[smallest])
                                smallest = right;
                        if(smallest != index)
```

```
                        {
                                swap(&heap[index],&heap[smallest]);
                                index = smallest;
                        }
                        else{
                                break;
                        }
                }
}
void deleteMin()
        {
        if(size == 0)
                return;
        heap[0] = heap[size-1];
        size--;
        heapifyDown(0);
}
void printHeap()
{
        for(int i=0;i<size;i++)
                {
                        printf("%d ",heap[i]);
                }
}
int main()
{
        int n,del,val;
        scanf("%d",&n);
        for(int i=0;i<n;i++)
                {
                        scanf("%d",&val);
                        insert(val);
                }
        printHeap();
        scanf("%d\n",&del);
        for(int i=0;i<del;i++)
                {
                        deleteMin();
                }
        printHeap();
        printf("\n");
        return 0;
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
| --- |
| **User Output** |
| 5 |
| 20 15 30 5 10 |
| 2 |
| 5 10 30 20 15 |
| 15 20 30 |

| Test Case - 2 |
| --- |
| **User Output** |
| 8 |
| 70 10 90 20 60 80 30 40 |
| 4 |
| 10 20 30 40 60 90 80 70 |
| 60 70 80 90 |

| S.No: 4 | Exp. Name: ***Max Heap Operations*** | Date: 2026-09-03 |
|---|---|---|

**Aim:**

Write a C program to implement a Max Heap. Insert a given list of integers into the heap while maintaining the max-heap property. Then, delete the root element (i.e., the maximum element) a specified number of times. Display the heap after all insertions and again after the deletions.

**Input Format:**

- The first line contains an integer $n$, the number of elements to insert into the heap.
- The second line contains $n$ space-separated integers representing the values to be inserted.
- The third line contains an integer $k$, the number of times the root element (maximum) should be deleted.

**Output Format:**

- The first line displays the space-separated heap contents after all insertions.
- The second line displays the space-separated heap contents after performing $k$ deletions of the root element.

**Note:** Refer to the test cases for better understanding.

**Source Code:**

maxheap.c

```
// write your code here...
#include <stdio.h>
#define  MIN_SIZE 20
int heap[MIN_SIZE];
int size = 0;
void swap(int*a,int*b)
{
        int temp = *a;
        *a = *b;
        *b = temp;
}
void heapifyup(int index)
{
        while (index > 0)
                {
                        int parent = (index - 1)/2;
                        if(heap[index]>heap[parent])
                        {
                                swap(&heap[index],&heap[parent]);
                                index = parent;
                        }
                        else{
                                break;
                        }
                }
}
void insert(int value)
{
        heap[size] = value;
        heapifyup(size);
        size++;
}
void heapifyDown(int index)
{
        while(1)
                {
                        int left = (2*index)+1;
                        int right = (2*index)+2;
                        int highest = index;
                        if(left<size&&heap[left]>heap[highest])
                                highest = left;
                        if(right<size&&heap[right]>heap[highest])
                                highest=right;
                        if(highest !=index)
```

```
                        {
                                swap(&heap[index],&heap[highest]);
                                index=highest;
                        }
                        else{
                                break;
                        }
                }
}
void deleMax()
{
        if(size==0)return;
        heap[0]=heap[size-1];
        size--;
        heapifyDown(0);
}
void printHeap()
{
        for(int i=0;i<size;i++)
                {
                        printf("%d ",heap[i]);
                }
}
int main()
{
        int n,del,val;
        scanf("%d",&n);
        for(int i=0;i<n;i++)
                {
                        scanf("%d",&val);
                        insert(val);
                }
        printHeap();
        scanf("%d\n",&del);
        for(int i=0;i<del;i++)
                {
                        deleMax();
                }
        printHeap();
        printf("\n");
        return 0;
}
```

**Execution Results** - All test cases have succeeded!

| Test Case - 1 |
| --- |
| **User Output** |
| 6 |
| 30 10 20 40 50 5 |
| 2 |
| 50 40 20 10 30 5 |
| 30 10 20 5 |

| Test Case - 2 |
| --- |
| **User Output** |
| 4 |
| 8 3 6 2 |
| 1 |
| 8 3 6 2 |
| 6 3 2 |

ID: 25B01A1228



Shri Vishnu Engineering College for Women (Autonomous)

| S.No: 5 | Exp. Name: ***Depth First Search (DFS) on a Graph*** | **Date: 2026-09-03** |
|---|---|---|

**Aim:**

Write a program to perform Depth First Search (DFS) traversal on an undirected graph using an adjacency matrix representation. The program should take the number of vertices, edges, and the starting vertex as input and print the DFS traversal of the graph.

**Input Format:**

- The first line contains an integer $N$, representing the number of vertices in the graph.
- The second line contains an integer $E$, representing the number of edges in the graph.
- The next $E$ lines contain two integers $v_1$, $v_2$ representing an edge between vertices $v_1$ and $v_2$.
- The last line contains an integer $S$, representing the starting vertex for the DFS traversal.

**Output Format:**

- Print the DFS traversal of the graph starting from $S$, with vertices separated by a space.

**Note:**

- If multiple nodes are available to visit, DFS should proceed in the order of their appearance in the adjacency matrix.

**Source Code:**

CTC64641.c

```
#include<stdio.h>
#define MAX 10
int adj[MAX][MAX];
int visited[MAX];
int stack[MAX];
int top = -1;
int n;
void push(int v)
{
        stack[++top] = v;
}
int pop()
{
        return stack[top--];
}
int isEmpty()
{
        return (top== -1);
}
void dfs_stack(int start)
{
        push(start);
        visited[start] = 1;
        while(!isEmpty())
                {
                        int current = pop();
                        printf("%d ",current);
                        for(int i=n-1;i>=0;i--)
                                {
                                        if(adj[current]
[i]==1&&!visited[i])
                                        {
                                                push(i);
                                        visited[i] = 1;
                                        }
                                }
                }
}
int main()
{
        int edges,u,v,start;
        scanf("%d",&n);
        scanf("%d",&edges);
        for(int i=0;i<n;i++)
```

```
                {
                        visited[i]=0;
                        for(int j=0;j<n;j++)
                                {
                                        adj[i][j]=0;
                                }
                }
        for(int i=0;i<edges;i++)
                {
                        scanf("%d%d",&u,&v);
                        adj[u][v]=1;
                        adj[v][u]=1;
                }
        scanf("%d",&start);
        dfs_stack(start);
        return 0;
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
|---|
| **User Output** |
| 4 |
| 4 |
| 0 1 |
| 0 2 |
| 1 2 |
| 2 3 |
| 0 |
| 0 1 2 3 |

| Test Case - 2 |
|---|
| **User Output** |
| 6 |
| 4 |
| 0 1 |
| 0 2 |

| |
|---|
| 3 4 |
| 4 5 |
| 0 |
| 0 1 2 |

| S.No: 6 | Exp. Name: ***Breadth First Search (BFS) on a Graph*** | **Date: 2026-09-17** |
|---|---|---|

**Aim:**

Write a program to perform Breadth First Search (BFS) traversal on an undirected graph using an adjacency matrix representation. The program should take the number of vertices, edges, and the starting vertex as input and print the BFS traversal of the graph.

**Input Format:**

- The first line contains an integer $N$, representing the number of vertices in the graph.
- The second line contains an integer $E$, representing the number of edges in the graph.
- The next $E$ lines contain two space-separated integers $v_1$, $v_2$ representing an edge between vertices $v_1$ and $v_2$.
- The last line contains an integer $S$, representing the starting vertex for the BFS traversal.

**Output Format:**

- Print the BFS traversal of the graph starting from $S$, with vertices separated by a space. If a vertex has multiple neighbors, process them in increasing order.

**Source Code:**

CTC64721.c

```
#include <stdio.h>
#include<stdlib.h>
#define MAX 10
int adj[MAX][MAX];
int visited[MAX];
int queue[MAX];
int front=0,rear=0;
int n;
void enqueue(int vertex)
{
        queue[rear++]=vertex;
}
int dequeue()
{
        return queue[front++];
}
int isEmpty()
{
        return front == rear;
}
void bfs(int start)
{
        visited[start]=1;
        enqueue(start);
        while(!isEmpty())
                {
                        int current = dequeue();
                        printf("%d ",current);
                        for(int i=0;i<n;i++)
                                {
                                        if(adj[current]
[i]==1&&!visited[i])
                                        {
                                                visited[i]=1;
                                                enqueue(i);
                                        }
                                }
                }
}
int main()
{
        int edges,u,v,start;
        scanf("%d",&n);
        scanf("%d",&edges);
```

```
        for(int i=0;i<n;i++)
                {
                        visited[i]=0;
                        for(int j=0;j<n;j++)
                                {
                                        adj[i][j]=0;
                                }
                        for(int i=0;i<edges;i++)
                                {
                                        scanf("%d%d",&u,&v);
                                        adj[u][v]=1;
                                        adj[v][u]=1;
                                }
                        scanf("%d",&start);
                        bfs(start);
                        return 0;
                }
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
|---|
| **User Output** |
| 4 |
| 4 |
| 0 1 |
| 0 2 |
| 1 2 |
| 2 3 |
| 0 |
| 0 1 2 3 |

| Test Case - 2 |
|---|
| **User Output** |
| 6 |
| 4 |
| 0 1 |

| 3 4 |
| --- |
| 4 5 |
| 0 |
| 0 1 2 |

| Test Case - 3 |
| --- |
| **User Output** |
| 1 |
| 0 |
| 0 |
| 0 |

| S.No: 7 | Exp. Name: ***Depth-First Search (DFS)*** | **Date: 2026-11-03** |
|---|---|---|

**Aim:**

Write a C program to implement and perform a Depth-First Search (DFS) on a directed graph represented using an adjacency list. The program should print all the nodes reachable from a given starting node.

**Input Format:**

- The first line contains two integers: $V$ (number of vertices) and $E$ (number of edges).
- The next $E$ lines each contain two integers: $u$ and $v$, where there is a directed edge from node $u$ to node $v$.
- The last line contains the starting node for the DFS traversal.

**Output Format:**

- Print all reachable nodes from the starting node in the order they are visited by DFS as a single line of space-separated integers.

**Note:**

- In DFS, the smaller numbered node is visited first when there are multiple adjacent nodes, ensuring a consistent traversal order
- Refer to the visible test cases for a better understanding.

**Source Code:**

dfs_directed_graph.c

```
//Write your code here..
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
// Adjacency list node
typedef struct Node {
 int vertex;
 struct Node* next;
} Node;
Node* adj[MAX]; // Adjacency list
int visited[MAX]; // Visited array
int n, e; // Number of vertices and edges
// Function to create a new node
Node* createNode(int v) {
 Node* newNode = (Node*)malloc(sizeof(Node));
 newNode->vertex = v;
 newNode->next = NULL;
 return newNode;
}
// Insert in sorted order (ascending)
void insertSorted(int u, int v) {
 Node* newNode = createNode(v);
 if (adj[u] == NULL || adj[u]->vertex > v) {
 newNode->next = adj[u];
 adj[u] = newNode;
 } else {
 Node* current = adj[u];
 while (current->next != NULL && current->next->vertex < v)
 current = current->next;
 newNode->next = current->next;
 current->next = newNode;
 }
}
// DFS function
void DFS(int v) {
 Node* temp;
 visited[v] = 1;
 printf("%d ", v);
 temp = adj[v];while (temp != NULL) {
 if (!visited[temp->vertex])
 DFS(temp->vertex);
 temp = temp->next;
 }
}
```

```
int main() {
 int i, u, v, start;
 // Read vertices and edges
 scanf("%d %d", &n, &e);
 // Initialize visited and adjacency list
 for (i = 0; i < n; i++) {
 visited[i] = 0;
 adj[i] = NULL;
 }
 // Read edges
 for (i = 0; i < e; i++) {
 scanf("%d %d", &u, &v);
 insertSorted(u, v); // Directed edge u ? v
 }
 // Read starting node
 scanf("%d", &start);
 // Perform DFS
 DFS(start);
 return 0;
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
|---|
| **User Output** |
| 5 6 |
| 0 1 |
| 0 2 |
| 1 3 |
| 1 4 |
| 2 4 |
| 3 4 |
| 0 |
| 0 1 3 4 2 |

| Test Case - 2 |
|---|
| **User Output** |
| 5 5 |

| 0 1 |
| --- |
| 0 4 |
| 1 2 |
| 2 3 |
| 3 4 |
| 0 |
| 0 1 2 3 4 |

| S.No: 8 | Exp. Name: ***Breadth First Search (BFS)*** | Date: 2026-11-03 |
|---|---|---|

**Aim:**

Write a C program to perform BFS on a directed graph using adjacency lists. The graph has $V$ vertices and $E$ edges. After building the graph, sort each vertex's adjacency list in ascending order. Then perform BFS from a given starting node and print all reachable nodes in the order they are visited as space-separated integers. When multiple adjacent nodes are available, always visit the smallest one first.

**Input Format:**

- The input begins with a line containing two integers $V$ and $E$, where $V$ is the number of vertices and $E$ is the number of edges in the directed graph.
- The following $E$ lines each contain two integers $u$ and $v$, indicating a directed edge from vertex $u$ to vertex $v$.
- The final line contains a single integer $start$, which represents the starting vertex for the BFS traversal.

**Output Format:**

- Print all reachable nodes from the starting vertex in the order they are visited by BFS, as a single line of space-separated integers.

**Note:** Refer to the test cases for better understanding.

**Source Code:**

BFS.c

```
// write your code here..
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
// Adjacency list node
typedef struct Node {
 int vertex;
 struct Node* next;
} Node;
Node* adj[MAX]; // Adjacency list
int visited[MAX]; // Visited array
int queue[MAX]; // Queue for BFS
int front = 0, rear = 0;
int n, e; // Number of vertices and edges
// Function to create a new node
Node* createNode(int v) {
 Node* newNode = (Node*)malloc(sizeof(Node));
 newNode->vertex = v;
 newNode->next = NULL;
 return newNode;
}
// Enqueue operation
void enqueue(int v) {
 queue[rear++] = v;
}
// Dequeue operation
int dequeue() {
 return queue[front++];
}
// Insert into adjacency list in ascending order
void insertSorted(int u, int v) {
 Node* newNode = createNode(v);
 if (adj[u] == NULL || adj[u]->vertex > v) {
 newNode->next = adj[u];
 adj[u] = newNode;
 } else {
 Node* current = adj[u];
 while (current->next != NULL && current->next->vertex < v)current =
current->next;
 newNode->next = current->next;
 current->next = newNode;
 }
}
// BFS function
```

```
void BFS(int start) {
 int i;
 Node* temp;
 for (i = 0; i < n; i++)
 visited[i] = 0;
 enqueue(start);
 visited[start] = 1;
 while (front < rear) {
 int current = dequeue();
 printf("%d ", current);
 temp = adj[current];
 while (temp != NULL) {
 if (!visited[temp->vertex]) {
 enqueue(temp->vertex);
 visited[temp->vertex] = 1;
 }
 temp = temp->next;
 }
 }
}
int main() {
 int i, u, v, start;
 // Read number of vertices and edges
 scanf("%d %d", &n, &e);
 // Initialize adjacency list
 for (i = 0; i < n; i++) {
 adj[i] = NULL;
 }
 // Read edges
 for (i = 0; i < e; i++) {
 scanf("%d %d", &u, &v);insertSorted(u, v); // Directed edge u ? v
 }
 // Read starting vertex
 scanf("%d", &start);
 // Perform BFS
 BFS(start);
 return 0;
}
```

## Execution Results - All test cases have succeeded!

**Test Case - 1**

| User Output |
| --- |
| 5 4 |
| 0 1 |
| 0 2 |
| 1 3 |
| 1 4 |
| 0 |
| 0 1 2 3 4 |

| Test Case - 2 |
| --- |
| **User Output** |
| 4 6 |
| 0 1 |
| 0 2 |
| 0 3 |
| 1 2 |
| 1 3 |
| 2 3 |
| 0 |
| 0 1 2 3 |

| S.No: 9 | Exp. Name: ***Bi-Connected Components in Graph*** | **Date: 2026-11-03** |
|---|---|---|

**Aim:**

Write a C program for finding the count of bi-connected components in a given graph

**Input Format:**

- First line in the input indicates the number of vertices.
- Second line in the input indicates the number of edges.
- The next following lines indicates the edges, which takes two vertices $u$ $v$ as space separated integers for each edge.

**Output Format:**

- The code will print the number of biconnected components found in the graph.

**Note:** Refer to the test cases for better understanding.

**Source Code:**

biConnected.c

```
// write your code here...
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
// Stack to store edges during DFS
typedef struct {
 int u, v;
} Edge;
Edge stack[MAX * MAX];
int top = -1;
void push(int u, int v) {
 stack[++top].u = u;
 stack[top].v = v;
}
Edge pop() {
 return stack[top--];
}
int disc[MAX], low[MAX], parent[MAX];
int time_dfs = 0;
int bccCount = 0;
void BCCUtil(int u, int adj[MAX][MAX], int V) {
 int children = 0;
 disc[u] = low[u] = ++time_dfs;
 for (int v = 0; v < V; v++) {
 if (adj[u][v]) {
 if (disc[v] == -1) {
 children++;
 parent[v] = u;
 push(u, v);
 BCCUtil(v, adj, V);
 low[u] = (low[u] < low[v]) ? low[u] : low[v];
 // If u is an articulation point
 if ((disc[u] == 1 && children > 1) || (disc[u] > 1 &&
low[v] >= disc[u])) {
 bccCount++;
while (1) {
 Edge e = pop();
 if (e.u == u && e.v == v)
 break;
 }
 }
 }
 else if (v != parent[u] && disc[v] < disc[u]) {
 low[u] = (low[u] < disc[v]) ? low[u] : disc[v];
```

```
 push(u, v);
 }
 }
 }
}
void BCC(int adj[MAX][MAX], int V) {
 for (int i = 0; i < V; i++) {
 disc[i] = -1;
 low[i] = -1;
 parent[i] = -1;
 }
 for (int i = 0; i < V; i++) {
 if (disc[i] == -1) {
 BCCUtil(i, adj, V);
 int flag = 0;
 while (top != -1) {
 pop();
 flag = 1;
 }
 if (flag)
 bccCount++;
 }
 }
}
int main() {
 int V, E;
 scanf("%d", &V);
 scanf("%d", &E);
 int adj[MAX][MAX] = {0};
 for (int i = 0; i < E; i++) {
 int u, v;scanf("%d %d", &u, &v);
 adj[u][v] = 1;
 adj[v][u] = 1;
 }
 BCC(adj, V);
 printf("%d\n", bccCount);
 return 0;
}
```

## Execution Results - All test cases have succeeded!

**Test Case - 1**

| User Output |
| --- |
| 3 |
| 2 |
| 0 1 |
| 0 2 |
| 2 |

| Test Case - 2 |
| --- |
| **User Output** |
| 5 |
| 4 |
| 0 1 |
| 1 2 |
| 1 3 |
| 2 4 |
| 4 |

| S.No: 10 | Exp. Name: ***C program that implements the quick sort*** | **Date: 2026-07-23** |
|---|---|---|

**Aim:**

Write a C program that implements the quick sort to sort a given list of integers in ascending order.

**Input Format**

- The first line contains an integer $n$ which represents the number of elements in the array.
- The second line contains $n$ space-separated integers, which are the elements of the array.

**Output Format**

- A single line containing $n$ space-separated integers, representing the array elements in ascending order after sorting.

**Note:**Refer to the test cases for better understanding.

**Source Code:**

quickSort.c

```
// write your code here...
#include<stdio.h>
void swap(int*a,int*b)
{
        int temp = *a;
        *a = *b;
        *b = temp;
}
int partition(int arr[],int low,int high)
{
int pivot=arr[low];
int i = low;
int j = high+1;
while(i < j){
                do{
                        i++;
                }while(arr[i]<=pivot);
                do{
                        j--;
                }while(arr[j]>pivot);
                if(i<j){
                        swap(&arr[i],&arr[j]);
        }
}
swap(&arr[low],&arr[j]);
return j;
}
void quicksort(int arr[],int low,int high)
{
        if(low<high)
        {
                int pi=partition(arr,low,high);
                quicksort(arr,low,pi-1);
                quicksort(arr,pi+1,high);
        }
}
        int main()
        {
                int n,i;
                scanf("%d",&n);
                int arr[n];
                for(i=0;i<n;i++)
                        {
                                scanf("%d",&arr[i]);
```

```
                }
        quicksort(arr,0,n-1);
        for(i=0;i<n;i++){
                printf("%d ",arr[i]);
}
printf("\n");
return 0;
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
|---|
| **User Output** |
| 6 |
| 6 2 8 10 36 14 |
| 2 6 8 10 14 36 |

| Test Case - 2 |
|---|
| **User Output** |
| 8 |
| 95 14 10 23 36 2 5 35 |
| 2 5 10 14 23 35 36 95 |

| S.No: 11 | Exp. Name: ***C program that implements the merge sort*** | **Date: 2026-07-30** |
|---|---|---|

**Aim:**

Write a C program to implement the Merge Sort algorithm to sort an array of integers in ascending order.

**Input Format**

- The first line contains an integer $n$ which represents the number of elements in the array.
- The second line contains $n$ space-separated integers, which are the elements of the array.

**Output Format**

- A single line containing $n$ space-separated integers, representing the array elements in ascending order after sorting.

**Note:** Refer to the test cases for better understanding.

**Source Code:**

mergeSort.c

```
// write your code here...
#include<stdio.h>
void merge(int arr[],int low,int mid,int high)
{
        int temp[10];
        int i= low;
        int j= mid + 1;
        int k = low;
        while(i <= mid && j <= high)
                {
                        if(arr[i] <= arr[j])
                        {
                                temp[k++]=arr[i++];
                        }
                        else
                        {
                                temp[k++]=arr[j++];
                }
                }
        while(i<=mid)
        {
                temp[k++]=arr[i++];
        }
        while(j<=high)
                {
                temp[k++]=arr[j++];
                }
        for(i=low;i<=high;i++)
        {
                arr[i]=temp[i];
        }
}
void mergesort(int arr[],int low,int high)
{
        if(low < high)
        {
                int mid = (low + high)/2;
                mergesort(arr,low,mid);
                mergesort(arr,mid+1,high);
                merge(arr,low,mid,high);
        }
}
int main()
{
```

```
        int n,i;
        scanf("%d",&n);
        int arr[n];
        for(i=0;i<n;i++)
        {
                scanf("%d",&arr[i]);
        }
        mergesort(arr,0,n-1);
        for(i=0;i<n;i++)
                {
                        printf("%d ",arr[i]);
                }
        printf("\n");
        return 0;
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
|---|
| **User Output** |
| 5 |
| 50 20 30 40 10 |
| 10 20 30 40 50 |

| Test Case - 2 |
|---|
| **User Output** |
| 10 |
| 25 78 15 45 32 65 46 38 76 12 |
| 12 15 25 32 38 45 46 65 76 78 |

| S.No: 12 | Exp. Name: ***Shortest Path from Source to Vertex*** | **Date: 2026-11-03** |
|---|---|---|

**Aim:**

You are required to write a C program that computes the shortest paths from a source vertex to all other vertices in a weighted, directed graph. The program should utilize Dijkstra's algorithm to solve this problem.

**Input Format:**

- The first line contains two integers $V$ and $E$, representing the number of vertices and the number of edges respectively.
- The next $E$ lines each contain three integers $u$, $v$, and $w$, representing a directed edge from vertex $u$ to vertex $v$ with weight $w$.
- The last line contains a single integer $src$, representing the source vertex from which shortest distances are to be calculated.

**Output Format:**

- The output consists of $V$ lines.
- Each line contains two integers:
    - The vertex number and the minimum distance from the source vertex to that vertex, space separated

**Note:** Refer to the test cases for better understanding.

**Source Code:**

Path.c

```
// write your code here...
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>
#define V 1000 // maximum vertices
int d[V]; // distance array
bool visited[V];
int G[V][V]; // adjacency matrix
// Initialize distances
void singleSource(int n, int s) {
 int i;
 for (i = 0; i < n; i++) {
 d[i] = INT_MAX;
 visited[i] = false;
 }
 d[s] = 0;
}
// Relax operation
void relax(int u, int v, int w) {
 if (d[u] != INT_MAX && d[v] > d[u] + w) {
 d[v] = d[u] + w;
 }
}
// Extract vertex with minimum distance
int extractMin(int n) {
 int v;
 int min = INT_MAX, min_index = -1;
 for (v = 0; v < n; v++) {
 if (!visited[v] && d[v] < min) {
 min = d[v];
 min_index = v;
 }
 }
 return min_index;
}
// Dijkstra’s algorithm
void dijkstra(int n, int s) {
 int count, u, v;
 singleSource(n, s);for (count = 0; count < n - 1; count++) {
 u = extractMin(n);
 if (u == -1) break; // no more reachable vertices
 visited[u] = true;
 for (v = 0; v < n; v++) {
 if (G[u][v] != 0 && !visited[v]) {
```

```
 relax(u, v, G[u][v]);
 }
 }
 }
}
int main() {
 int n, e;
 int i, j;
 int u, v, w;
 int s;
 scanf("%d %d", &n, &e);
 // initialize graph
 for (i = 0; i < n; i++) {
 for (j = 0; j < n; j++) {
 G[i][j] = 0;
 }
 }
 // read edges
 for (i = 0; i < e; i++) {
 scanf("%d %d %d", &u, &v, &w);
 G[u][v] = w; // directed edge
 }
 scanf("%d", &s);
 dijkstra(n, s);
 // print result
 for (i = 0; i < n; i++) {
 if (d[i] == INT_MAX)
 printf("%d INF\n", i);
 else
 printf("%d %d\n", i, d[i]);
 }return 0;
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
|---|
| **User Output** |
| 4 3 |
| 0 1 2 |
| 0 2 5 |
| 0 3 1 |

| 0 |
|---|
| 0 0 |
| 1 2 |
| 2 5 |
| 3 1 |

| Test Case - 2 |
|---|
| **User Output** |
| 6 9 |
| 0 1 7 |
| 0 2 9 |
| 0 5 14 |
| 1 2 10 |
| 1 3 15 |
| 2 3 11 |
| 2 5 2 |
| 3 4 6 |
| 5 4 9 |
| 0 |
| 0 0 |
| 1 7 |
| 2 9 |
| 3 20 |
| 4 20 |
| 5 11 |

| S.No: 13 | Exp. Name: ***Job Sequencing using Greedy Problem*** | **Date: 2026-11-03** |
|---|---|---|

**Aim:**

Given a set of$N$jobs, where eachjob $i$has a deadline and a profit associated with it. Each job takes1unit of time to complete, and only one job can be scheduled at a time. We earn the profit associated with the job if and only if the job is completed by its deadline. Find the number of jobs done and themaximum profit.

**Note:**Jobs will be given in the form (Job Id, Deadline, Profit) associated with that Job. The deadline of the job is the time by which the job needs to be completed to earn the profit.

**Input Format:**

- The first line contains an integer $N$ representing the number of jobs.
- The next $N$ lines each contain three space-separated integers: {Job ID, Job Deadline, Job Profit} — representing one job per line.

**Output Format**

- Two integers separated by a space:
- The first integer represents the number of jobs done.The second integer represents the maximum profit obtained.

**Constraints:**

- $1 \leq N \leq 10^5$
- $1 \leq Deadline \leq N$
- $1 \leq Profit \leq 500$

**Example 1:**

**Input:**

```
4
1 4 20
2 1 10
3 1 40
4 1 30
```

**Output:**

```
2 60
```

**Explanation:** Job 1and Job 3 can be done with a maximum profit of 60 (20+40).

**Note:** Refer to the test cases for better understanding.

**Source Code:**

JsGreedy.c

```
// write your code here...
#include <stdio.h>
#include <stdlib.h>
// Structure to store job details
struct Job {
 int id;
 int deadline;
 int profit;
};
// Compare function for sorting jobs by profit (descending)
int compare(const void *a, const void *b) {
 struct Job *j1 = (struct Job *)a;
 struct Job *j2 = (struct Job *)b;
 return j2->profit - j1->profit;
}
int main() {
 int n, i, j;
 scanf("%d", &n);
 struct Job jobs[n];
 int maxDeadline = 0;
 for (i = 0; i < n; i++) {
 scanf("%d %d %d", &jobs[i].id, &jobs[i].deadline,
&jobs[i].profit);
 if (jobs[i].deadline > maxDeadline) {
 maxDeadline = jobs[i].deadline;
 }
 }
 // Sort jobs by profit
 qsort(jobs, n, sizeof(struct Job), compare);
 // Array to keep track of free time slots
 int slot[maxDeadline + 1];
 for (i = 0; i <= maxDeadline; i++) {
 slot[i] = -1;
 }
 int countJobs = 0, maxProfit = 0;
 // Schedule jobs
        for (i = 0; i < n; i++) {
 // Find a free slot for this job (latest possible slot)
 for (j = jobs[i].deadline; j > 0; j--) {
 if (slot[j] == -1) {
 slot[j] = jobs[i].id; // Assign job
 countJobs++;
 maxProfit += jobs[i].profit;
 break;
```

```
  }
  }
  }
  printf("%d %d\n", countJobs, maxProfit);
  return 0;
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
|---|
| **User Output** |
| 4 |
| 1 4 20 |
| 2 1 10 |
| 3 1 40 |
| 4 1 30 |
| 2 60 |

| Test Case - 2 |
|---|
| **User Output** |
| 5 |
| 1 2 60 |
| 2 1 100 |
| 3 3 20 |
| 4 4 70 |
| 5 2 50 |
| 4 250 |

| S.No: 14 | Exp. Name: ***Program to implement Knapsack problem Using Dynamic Programming*** | **Date: 2026-11-03** |
|---|---|---|

**Aim:**

You are given $n$ items, each with a specific weight and value. You have a knapsack with a maximum capacity $W$. Your goal is to select a subset of these items such that the total weight is less than or equal to $W$, and the total value is as large as possible.

This is the classic 0/1 Knapsack problem, where each item can either be included once or excluded from the knapsack.

**Input Format:**

- The first line contains an integer $n$, representing the number of items to consider for the knapsack.
- The second line contains $n$ space-separated integers, representing the weights of the items.
- The third line contains $n$ space-separated integers, representing the values of the items, corresponding to their weights.
- The fourth line contains an integer $W$, representing the capacity of the knapsack.

**Output Format:**

- Print the maximum profit that can be obtained by selecting items such that their combined weight does not exceed the knapsack capacity.

**Note:** Refer to the test cases for better understanding.

**Source Code:**

dynamicKs.c

```
// write your code here...
#include <stdio.h>
// Function to return maximum of two numbers
int max(int a, int b) {
 return (a > b) ? a : b;
}
int main() {
 int n, W;
 int i, w;
 // Input number of items
 scanf("%d", &n);
 int weight[n], value[n];
 // Input weights
 for (i = 0; i < n; i++) {
 scanf("%d", &weight[i]);
 }
 // Input values
 for (i = 0; i < n; i++) {
 scanf("%d", &value[i]);
 }
 // Input knapsack capacity
 scanf("%d", &W);
 // DP table
 int dp[n + 1][W + 1];
 // Initialize DP table
 for (i = 0; i <= n; i++) {
 for (w = 0; w <= W; w++) {
 if (i == 0 || w == 0) {
 dp[i][w] = 0;
 } else if (weight[i - 1] <= w) {
 dp[i][w] = max(value[i - 1] + dp[i - 1][w - weight[i -
1]], dp[i - 1][w]);
 } else {
 dp[i][w] = dp[i - 1][w];
 }
 } }
 // Print maximum profit
 printf("%d\n", dp[n][W]);
 return 0;
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
| --- |
| **User Output** |
| 5 |
| 5 10 15 20 25 |
| 10 20 30 40 50 |
| 500 |
| 150 |

| Test Case - 2 |
| --- |
| **User Output** |
| 4 |
| 100 200 300 400 |
| 10 20 30 40 |
| 100 |
| 10 |

| S.No: 15 | Exp. Name: ***N-Queens Problem*** | **Date: 2026-11-03** |
|---|---|---|

**Aim:**

Write a C program to solve the N-Queens problem using backtracking. The goal is to place $N$ queens on an $N \times N$ chessboard such that no two queens attack each other. A queen can attack another queen if they share the same row, column, or diagonal. Your program should print the first valid configuration of the board found by placing queens from left to right and top to bottom, using a backtracking approach. The board must be displayed as an $N \times N$ matrix where a cell contains 1 if a queen is placed there, and 0 otherwise. If no solution exists for the given value of $N$, print a message indicating that no solution exists.

**Input Format:**

- A single integer $N$ — the number of queens and the size of the chessboard.

**Output Format:**

- If a solution exists, print an $N \times N$ matrix consisting of 0s and 1s, where 1 represents a queen.
- If no solution exists, print: "No solution exists for N = <value>".
- Each row of the matrix should be printed on a new line, with space-separated values.

**Note:** Refer to the test cases for better understanding.

**Source Code:**

nqueens.c

```
// write your code here...
#include <stdio.h>
int n;
int board[20][20];

int safe(int row, int col) {
    int i, j;
    for (i = 0; i < row; i++) {
        if (board[i][col] == 1)
            return 0;
    }
    for (i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j] == 1)
            return 0;
    }
    for (i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++) {
        if (board[i][j] == 1)
            return 0;
    }
    return 1;
}

int solve(int row) {
    if (row == n)
        return 1;
    for (int col = 0; col < n; col++) {
        if (safe(row, col)) {
            board[row][col] = 1;
            if (solve(row + 1))
                return 1;
            board[row][col] = 0;
        }
    }
    return 0;
}

int main() {
    scanf("%d", &n);
    if (solve(0)) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                printf("%d ", board[i][j]);
            }
            printf("\n");
```

```
        }
    } else {
        printf("No solution exists for N = %d\n", n);
    }
    return 0;
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
|---|
| **User Output** |
| 4 |
| 0 1 0 0 |
| 0 0 0 1 |
| 1 0 0 0 |
| 0 0 1 0 |

| Test Case - 2 |
|---|
| **User Output** |
| 3 |
| No solution exists for N = 3 |

| S.No: 16 | Exp. Name: ***Knapsack using backtracking*** | **Date: 2026-11-03** |
|---|---|---|

**Aim:**

Write a C program to solve the 0/1 Knapsack Problem using backtracking approach. The program aims to maximize the total value of items that can be included in a knapsack without exceeding its capacity

**Input Format:**

- The first line contains an integer $n$, representing the number of items.
- The next $n$ lines each contain two space-separated integers: the value and weight of an item.
- The last line contains an integer $W$, representing the capacity of the knapsack.

**Output Format:**

- Print the maximum profit (total value) that can be obtained by selecting items, considering the knapsack's capacity constraint.

**Note:** Refer to the test cases for better understanding.

**Source Code:**

knapsackBacktracking.c

```
// write your code here...
#include<stdio.h>
int maxProfit=0;
void knapsack(int i,int n,int capacity,int weight[],int value[],int
currWeight,int currValue)
{
    if(currWeight>capacity)
        return;
    if(currValue>maxProfit)
        maxProfit=currValue;
    if(i==n)
        return;

    knapsack(i+1,n,capacity,weight,value,currWeight +
weight[i],currValue + value[i]);
    knapsack(i+1,n,capacity,weight,value,currWeight,currValue);
}
int main(){
    int n,W;
    scanf("%d",&n);
    int value[n],weight[n];
    for(int i=0;i<n;i++){
        scanf("%d %d",&value[i],&weight[i]);
    }
    scanf("%d",&W);
    knapsack(0,n,W,weight,value,0,0);
    printf("%d\n",maxProfit);
    return 0;
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
| --- |
| **User Output** |
| 3 |
| 60 10 |
| 100 20 |
| 120 30 |
| 50 |

| |
|---|
| 220 |

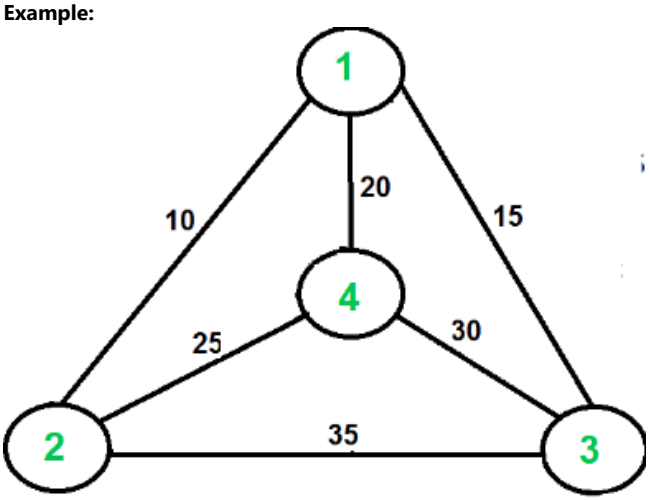
| Test Case - 2 |
|---|
| **User Output** |
| 5 |
| 10 5 |
| 40 4 |
| 30 3 |
| 50 2 |
| 20 1 |
| 100 |
| 150 |

| S.No: 17 | Exp. Name: ***Travelling Salesman Problem*** | **Date: 2026-11-03** |
|---|---|---|

**Aim:**

Given a set of cities and the distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point.

**Example:**




A TSP tour in the graph is 1-2-4-3-1. The cost of the tour is 10+25+30+15, which is 80

**Input Format:**

- The first line contains an integer $n$, representing the number of cities.
- The following $n$ lines contain the cost matrix of size $n \times n$. Each line contains $n$ integers separated by spaces, representing the cost of traveling from one city to another. If there is no direct connection between two cities, -1 is used to represent infinity.

**Output Format:**

- The output is a single integer representing the optimal cost of the traveling salesman problem.

**Source Code:**

CTC30966.c

```
#include<stdio.h>
#include<limits.h>
#define m 15
#define INF 9999999
int n;
int c[m][m];
int dp[1<< m][m];
int min(int a,int b){
    return (a<b)? a:b;
}
int tsp(int ma, int p){
    if(ma == (1<< n) -1)
        return c[p][0] == INF? INF:c[p][0];
    if(dp[ma][p] != -1)
        return dp[ma][p];
    int a = INF;
    for(int city = 0; city<n; city++){
        if((ma&(1 << city)) == 0 && c[p][city] != INF){
            int na = c[p][city] + tsp(ma| (1<<city),city);
            a = min(a,na);
        }
    }
    return dp[ma][p] = a;
}
int main(){
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            scanf("%d",&c[i][j]);
            if(c[i][j] == -1)
                c[i][j] = INF;
        }
    }
    for(int i=0;i<(1<<n);i++)
        for(int j=0;j<n;j++)
            dp[i][j] = -1;
    int a=tsp(1,0);
    if(a>INF)
        printf("-1\n");
    else
        printf("%d\n",a);
}
```

## Execution Results - All test cases have succeeded!

| Test Case - 1 |
| --- |
| **User Output** |
| 3 |
| -1 2 3 |
| 4 -1 6 |
| 1 2 -2 |
| 9 |

| Test Case - 2 |
| --- |
| **User Output** |
| 4 |
| -1 2 3 4 |
| 1 -1 1 3 |
| 4 5 -1 1 |
| 2 3 6 -1 |
| 6 |